

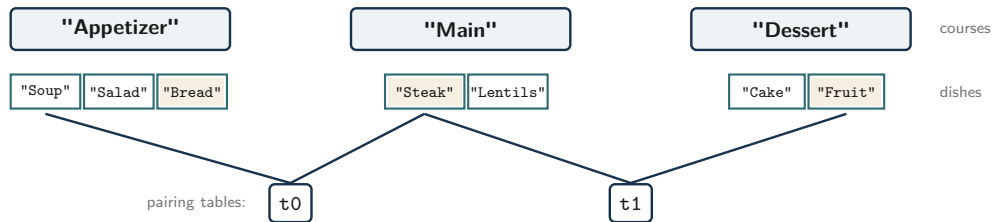
PA 5,6 - Hash Tables & Heaps - Naming, Tweaking, and Ranking Meals

Big idea: put Units 5 & 6 to work on the Menu Planner: give every course and dish its **name**, build **name maps** that spread those names well, then an **indexed heap** that powers **tweaking** and **ranking**.

Today

- ▶ a new `.names` file names every course and dish
- ▶ **Part 1 (Unit 5):** name maps that spread the names well
- ▶ **Part 2 (Unit 6):** an **indexed heap** for **tweaking** and **ranking**

The Menu Planner — In One Picture



$$\text{score} = t0(\text{"Bread"}, \text{"Steak"}) \times t1(\text{"Steak"}, \text{"Fruit"})$$

 the meal's picks

From PA 3,4: a **meal** picks one dish per course. Each **pairing table** scores the dishes of the courses it links; a meal's **score** is the **product** over the tables. A **chosen** course is locked to its dish in every meal.

The .names sidecar

NAMES

<numCourses>

<CourseName> <numDishes> <dishName0> ...

one line per course, in .menu course order

Single tokens (an underscore stands in for a space, e.g. Garlic_Bread), 2 to 24 characters of letters, digits, and underscores. Names are **distinct**: unique among the courses, and unique among one course's dishes. Counts must match the .menu.

samples/menu3.names

NAMES

3

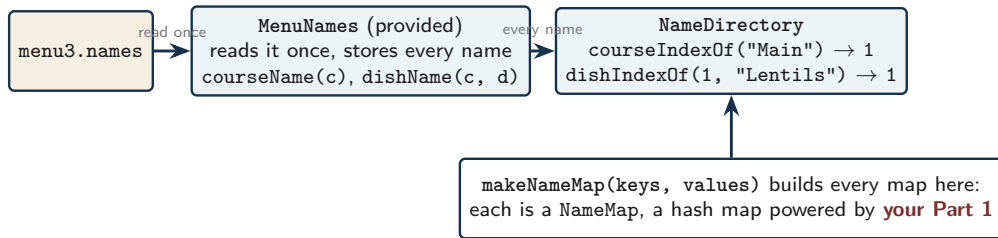
Appetizer 3 Soup Salad Bread

Main 2 Steak Lentils

Dessert 2 Cake Fruit

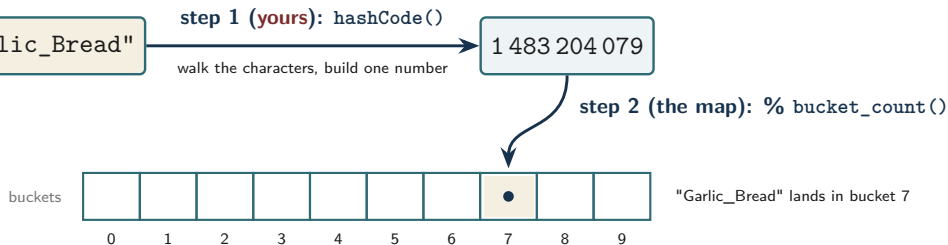
Parsing is **provided**: MenuNames.hpp serves `courseName(c)`, `dishName(c, d)`, and the counts. Your work starts where the name arrays end.

Given — From a File of Names to Fast Lookups



Name → index is the direction every request travels (a person says "Main", the model thinks course 1), so the lookups must be fast. That is what Part 1 builds.

From a Name to a Bucket



The map applies the final % for you. If you add a compression step of your own that needs a modulus, use `HASH_PRIME` from `IntegerHash.hpp`: a fixed prime far larger than any bucket count.

`std::unordered_map` takes a hash class of **YOUR** design

- ▶ `NameMap<K, V> = std::unordered_map<K, V, NameHasher>`: every name map carries *your* `NameHasher`.
- ▶ `makeNameMap(keys, values)` is the one place every name map is born. The shipped `NameHasher` **works**: every lookup succeeds.
- ▶ But it does **not** spread the names well enough, except on a few of the smallest name lists.

You may change **anything** you want in `NameHash.hpp`.

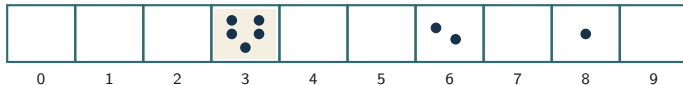
What your hasher must stay

- ▶ **Trivially copyable** and at most **32 bytes**, checked at *compile time* by the autograder.
- ▶ It **computes** its answer from the key's **characters** and a few stored **constants**. It may **not** store the names or any per-name data.
- ▶ **String keys are the exercise**. The **model's course-table index** rides the provided IntegerHasher (`IntegerHash.hpp`), a worked example worth reading.

What a Good Name Map Looks Like

piled up (8 names, 10 buckets)

● = one name



biggest bucket **5**, alone **1 of 8**

spread out (same 8 names)



biggest bucket **1**, alone **8 of 8**

A lookup computes one bucket and searches only it, so a crowd in one bucket is exactly what slows lookups down.

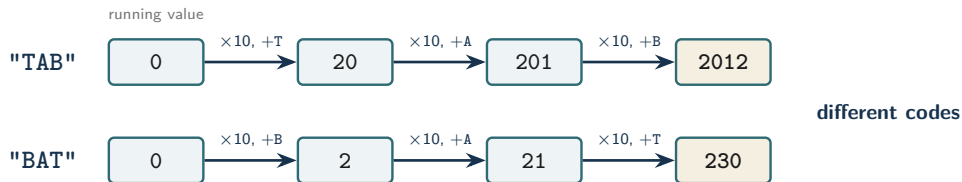
Spread is graded in steps: the autograder reports where you landed, and the provided driver prints the same report (biggest bucket, names alone).

A menu is printed once and read all evening

- ▶ `makeNameMap` builds each map **once**, with **exactly** `bucketCountFor(N)` buckets.
- ▶ There are always more buckets than names, so the map **never rehashes**. From then on it only answers lookups.
- ▶ No map holds more than `MAX_NAMES` (512) names, so `bucket_count()` never exceeds `MAX_BUCKET_COUNT` (1024).

The grade judges the **map you return**, never how your hash looks inside.

Order Matters — Scale, Then Add



(Letter values A=1, B=2, T=20; scale by 10.) The codes we built in class **scale, then add**, so the *order* of the characters matters. Just adding gives $20 + 1 + 2 = 23$ for **both** words. Menus are full of near-identical names, so a code that leans on a few characters, or ignores their order, gets into trouble.

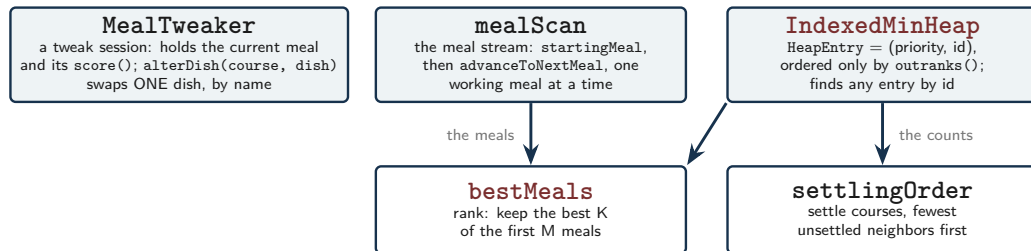
You write (in NameHash.hpp)

- ▶ `hashCode(std::string const&)` — step 1: turn a name into one number.
- ▶ `operator()(std::string const&)` — what the map calls on every key; the map reduces what it returns with `% bucket_count()`.

You use (provided)

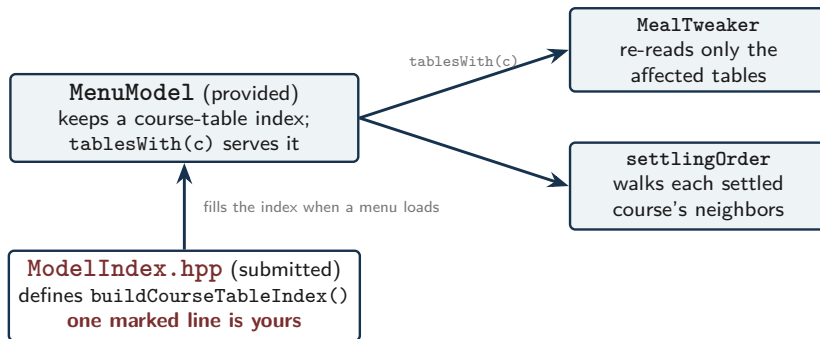
- ▶ `makeNameMap(keys, values)` — already written: builds every name map, presized to the required bucket count.
- ▶ `bucketCountFor(n)` — the exact `bucket_count()` your maps must have.
- ▶ `HASH_PRIME`, `mulmod(a, x, m)` — a big fixed prime and an overflow-safe $(a \cdot x) \bmod m$, for a compression that needs a modulus of its own. Optional.

Given — The Meal-Side Pieces



Part 2 in one look: you **build** the **IndexedMinHeap** and **bestMeals**; **MealTweaker**, **settlingOrder**, and the model's index-builder (next slide) arrive **written except one marked line each**. The next slides take these pieces one at a time.

The Model Serves Its Own Index



The split: `MenuModel.hpp` **declares** `buildCourseTableIndex()`; your `ModelIndex.hpp` **defines** it. You write one piece of the model without touching the rest. (Any file that constructs a `MenuModel` includes `ModelIndex.hpp`.)

You write (in ModelIndex.hpp)

- ▶ `MenuModel::buildCourseTableIndex()` — the shell is given; **one marked line** records table `t` on that course's list in `_tablesByCourse` (`[]` creates a course's empty list the first time).

You use

- ▶ `numTables()`, `factorAt(t)` — walk the pairing tables.
- ▶ `scopeSize()`, `courseAt(j)` — walk one table's courses.
- ▶ `tablesWith(c)` — how the finished index is SERVED to everyone (the tweaker reads it in Part 2).

The Locator — Why an Indexed Heap

the heap array (level order): priority on top, **id** below



the locator

id 2	→ slot 1
id 4	→ slot 3
id 6	→ slot 4
id 7	→ slot 0
id 9	→ slot 2

contains(id), priorityOf(id), and re-prioritizing are fast *because* the locator names the id's current slot. So **every swap your percolates make must update the locator too**. An array that is right with a locator that is stale fails the conformance tests.

Adapt the provided MinHeap (complete and working; do not submit it)

- ▶ `insert(priority, id)`: add the entry, or on an id **already** in the heap, **change its priority**; the entry moves to where it now belongs.
- ▶ `removeMin()` and `replaceMin(entry)`; `heapify(data)`, a static helper that is pure array work; `buildFrom(entries)`, after which the heap holds **exactly this batch**, locator included.
- ▶ `size` / `contains` / `priorityOf` / `peekMin` are **already written**: they simply report what `_data` and `_locator` say, so keep both accurate and they work by themselves.

RULE: every ordering decision between two entries goes through `HeapEntry::outranks()`. Smaller priority wins, equal priorities go to the smaller id.
Never compare priority fields directly.

You write (in IndexedMinHeap.hpp)

- ▶ `insert(priority, id)` — add a new entry; on an id **already** in the heap, **change** its priority instead (the entry moves to where it now belongs).
- ▶ `removeMin()` — remove and return the top entry; throws `std::out_of_range` when empty.
- ▶ `replaceMin(entry)` — swap the top entry for this one (non-empty heaps only).
- ▶ `heapify(data)` — static: turn any array of entries into a heap, in place. Pure array work; knows nothing about the locator.
- ▶ `buildFrom(entries)` — the heap becomes **exactly this batch**, locator included.
- ▶ your private percolate helpers — yours to design; start from MinHeap's `_percolateUp` / `_percolateDown`.

Already written for you (same file)

- ▶ `size()` — how many entries the heap holds.
- ▶ `contains(id)` — is this id in the heap?
- ▶ `priorityOf(id)` — that id's current priority; throws if the id is absent.
- ▶ `peekMin()` — look at the top entry without removing it; throws when empty.

Provided next door

- ▶ `HeapEntry::outranks(other)` — **the** comparison: smaller priority wins, ties go to the smaller id. Every ordering decision goes through it.
- ▶ `MinHeap (MinHeap.hpp)` — the complete plain heap you read and adapt.

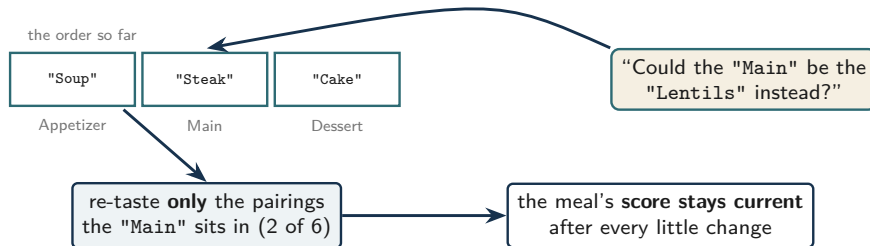
You write (in MenuIndex.hpp)

- ▶ **one marked line** inside `settlingOrder(model)`'s neighbor loop — a settled course's neighbor now has one fewer unsettled neighbor, so its count in the heap is out of date.

You use

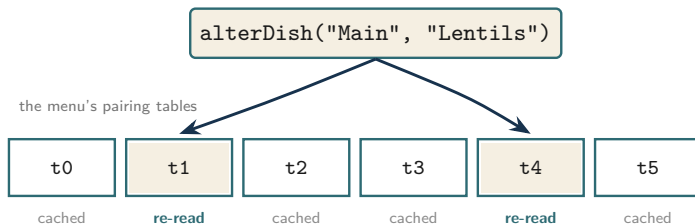
- ▶ `courseNeighbors(model)` — the neighbor lists, computed for you.
- ▶ `contains(id), priorityOf(id)` — ask your heap what it currently holds.
- ▶ `insert(priority, id)` — remember what it does on an id **already** in the heap.
- ▶ `courseIndexOf(name), dishIndexOf(courseIdx, name)` — the `NameDirectory` lookups, already written; -1 on a miss.

A Table Full of Small Adjustments



Patrons rarely reorder from scratch; they **adjust**: one dish at a time, again and again. That conversation is `MealTweaker`. It holds the table's current meal and its score, and each adjustment re-checks **only the pairings the changed course sits in**, never the whole menu.

One Dish Changes — Few Tables Care



```
tablesWith(1) = t1, t4  
(course 1 = "Main")
```

`alterDish` **refuses** a locked course and the already-selected dish (`false`, nothing changes); the names it is given always exist on the menu.

An accepted swap re-reads **only the swapped course's tables** (the model's `tablesWith(c)` knows which), then recombines the total from the cache. Why keep a whole cache instead of updating one total? The next three slides.

One Running Total — If No Score Were Ever 0

the six tables' scores



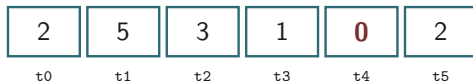
the swap changes t1: $5 \rightarrow 3$ and t4: $4 \rightarrow 1$



Then one number would be enough: **divide out** each affected table's old score, **multiply in** its new one. No cache at all. (You would still read the *old* values from the tables just before the swap, to know what to divide by.)

Then a Table Scores 0 — Divide-Out Breaks

one pairing scores 0



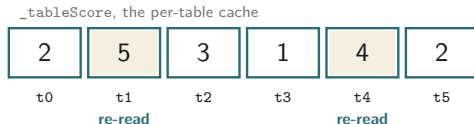
the swap fixes the pairing: t4: 0 $\rightarrow$ 4



true new score: $2 \cdot 5 \cdot 3 \cdot 1 \cdot 4 \cdot 2 = 240$

Once any factor is 0, the total is 0, and the other tables' values are **invisible inside it**: no division brings them back. The swap you most want, the one that escapes a 0 pairing, is **exactly the one divide-out cannot price**.

The Cache Answer — Keep Every Table's Score



the old values sit **in the cache**:
only the **new** ones are read (k reads, not $2k$)

recombine, a **fresh product** of the six cached numbers: $2 \cdot 5 \cdot 3 \cdot 1 \cdot 4 \cdot 2 = 240$ — nothing is ever divided

Zeros are **harmless**: a 0 just sits in its cache slot until a re-read replaces it, and every recombine multiplies fresh values (no divide, no leftover rounding error from repeated $\div$ and $\times$). A zero-counting patch can rescue divide-out, but it needs the old values *and* extra bookkeeping; the cache is simpler and costs one double per table.

MealTweaker — Functions at a Glance

You write (in MealTweaker.hpp)

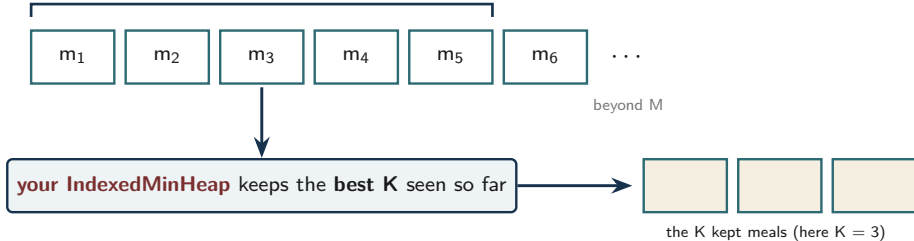
- ▶ **one marked line** inside `alterDish` — the refresh loop: which cache entries to re-read after the swap.

Already there / you use

- ▶ `score()`, `currentMeal()` — the cached total and the meal being tweaked.
- ▶ `_recombine()` — multiplies the cache into the total; already called for you.
- ▶ `model.isChosen(c)` — is this course locked?
- ▶ `model.tablesWith(c)` — the tables that involve course `c`, in one lookup (the index you built in `ModelIndex.hpp`).
- ▶ `tableScoreFor(model, t, meal)` — read ONE pairing table's score for a meal. **Every read is counted.**

Too Many Meals to Store

the first M , visited in order (mealScan.hpp, provided)



The meal space is the **product of the dish counts**: far too many meals to store, and too many to number with a single integer. So `bestMeals` visits the **first M** the menu can still realize and keeps the **best K** of them.

One sweep, K meals held

- ▶ Score each meal as it streams past, through the **one** reusable working Meal the scan hands you. Never a fresh Meal per candidate.
- ▶ Hold at most **K** Meal **objects** at any moment; **your IndexedMinHeap** decides which K stay.
- ▶ M caps meals **visited**. If the menu can realize fewer than M, you simply see them all.
- ▶ Return the K best, **best first**. Fewer than K only when fewer meals were seen.

Locked courses keep their chosen dish in **every** meal the scan visits.

You write (in BestMeals.hpp)

- ▶ `bestMeals(model, K, M)` — keep the best K of the first M meals visited; return them best first. (The loop shell is already written.)

You use

- ▶ `startingMeal` / `advanceToNextMeal` — set up the first meal (locked courses pinned), then step meal to meal; false when done.
- ▶ `mealScore(model, working)` — the meal's total score.
- ▶ `peekMin()` — your heap's view of the **worst kept** meal, the one a newcomer must beat.
- ▶ `outranks(...)` — the strict, tie-safe way to decide *beats*.
- ▶ plus whichever other IndexedMinHeap operations your design needs.

Three gates first — each can zero everything

- ▶ **Compile gate:** your six files, against the provided headers.
- ▶ **Memory gate:** everything runs under the address sanitizer; **any leak or memory error scores 0.**
- ▶ **Source policy:** `<unordered_map>`, `<vector>`, `<string>` and friends are *allowed*; the other STL containers, smart pointers, the library sorting and heap algorithms, using namespace, and **naming any instructor counter** are **banned**. Full lists in the instructions.

How It's Graded — The 50 Points

Where the points live

What	pts
Model tests (13×2): your functions' <i>return values</i> vs the reference	26
Unit groups (10×2): 4 name-map, 5 IndexedMinHeap, 1 meal edge cases	20
Performance (2 entries): an efficient organization of the work	4

Every group and test grades what your code **returns** or what the map/heap **measurably does**, never how your source looks. Know your own code: you may be **asked about your implementation** in class.

Submit exactly SIX files (Gradescope)

NameHash.hpp, ModelIndex.hpp, IndexedMinHeap.hpp, MenuIndex.hpp, MealTweaker.hpp, BestMeals.hpp. Filenames must match **exactly**. Three carry real building; ModelIndex.hpp, MenuIndex.hpp, and MealTweaker.hpp arrive finished except **one marked line each**. Resubmit as often as you like.

Build and test locally

`g++ -std=c++20 -Wall -Wextra -o driver driver.cpp`, then `./driver samples/menu3`. The driver prints each map's **spread**, the lookups, a tweak script with a self-check, the best meals, the settling order, and the work counters. **Compare against** `expected_output.txt` (its header marks the few design-dependent lines). Check for leaks with `-fsanitize=address,undefined`.

Grades run on **Ubuntu 22.04**, **g++ 11**, **-std=c++20**. Make sure all six files compile cleanly there.

Takeaways

- ▶ Every course and dish now has a **name**, and your NameMaps answer every name lookup, with the spread judged through the map's own bucket interface.
- ▶ An **indexed heap** finds and **re-prioritizes** any entry by id; every swap keeps the **locator** right, and every comparison goes through outranks().
- ▶ The model serves the **course-table index you built**; the tweaker re-reads only the swapped course's tables, and the ranking sweeps an enormous meal space holding at most **K meals**.

Class Question

What did it take to spread the names well, and what does the locator buy your heap?
Be ready to explain both, and *why*.